

E-Cart Analytics: Modernización hacia un Data Lakehouse en Databricks

Propósito del Proyecto: Este repositorio resuelve el caso de negocio "*Modernización de la Plataforma de Datos - E-Cart Analytics*". Su objetivo principal es migrar la carga analítica de una base de datos transaccional (PostgreSQL) que estaba sufriendo problemas de rendimiento, hacia una arquitectura moderna, escalable y gobernada en la nube utilizando **AWS** y **Databricks Unity Catalog**.

1. Contexto del Negocio (Para perfiles no técnicos)

El Problema Original: La empresa "E-Cart" operaba todo su comercio electrónico sobre una base de datos PostgreSQL. Con el rápido crecimiento de la empresa, los equipos de negocio (Finanzas y Marketing) comenzaron a ejecutar reportes pesados directamente sobre esta base de datos transaccional. Esto causaba caídas del sistema, enlentecía la aplicación para los usuarios finales y no permitía hacer análisis predictivos.

La Solución: Diseñar un **Data Lakehouse**. Extraer la información desde PostgreSQL sin afectar su rendimiento y llevarla a la nube (AWS S3). Una vez allí, utilizar el poder de procesamiento distribuido de **Databricks (Apache Spark)** para limpiar, transformar y organizar los datos, preparándolos para ser consumidos por tableros estadísticos (Power BI, Tableau) de manera súper rápida e independiente de la aplicación principal.

Extra Mile: De un concepto teórico a una plataforma funcional

El reto pedía *diseñar e implementar conceptualmente* la solución. Sin embargo, para demostrar habilidades de ingeniería reales, **este repositorio contiene un pipeline 100% funcional**. Como no se proveyeron datos de prueba, se desarrolló un simulador avanzado en Python ([bronze/generate_dummy_data.py](#)) utilizando la librería **Faker** para generar decenas de miles de registros sintéticos (con nombres latinos, correos y transacciones coherentes), permitiendo probar la ingesta y transformación en un entorno de Big Data realista.

```

generate_dummy_data.py 2, M X
spinbyoxxo-data-pipeline > bronze > generate_dummy_data.py > ...
6
7 # Configurar Faker para generar nombres y datos en español latino
8 fake = Faker('es_MX')
9
10 NUM_USERS = 10000
11 NUM_ORDERS = 30000
12
13 print("Generando datos para E-Card Analytics...")
14
15 # ----- Tabla USERS -----
16 users_data = []
17 countries = ['GT', 'SV', 'HN', 'CR', 'CO', 'MX', 'CL'] # Países de la región
18
19 for _ in range(NUM_USERS):
20     created = fake.date_time_between(start_date='-2y', end_date='-1m')
21     updated = created + timedelta(days=random.randint(0, 30))
22
23     users_data.append({
24         'id': str(uuid.uuid4()),
25         'name': fake.name(),
26         'email': fake.unique.email(),
27         'country': random.choice(countries),
28         'created_at': created.strftime('%Y-%m-%d %H:%M:%S'),
29         'updated_at': updated.strftime('%Y-%m-%d %H:%M:%S')
30     })
31
32 df_users = pd.DataFrame(users_data)
33
34 # ----- Tabla ORDERS -----
35 orders_data = []

```

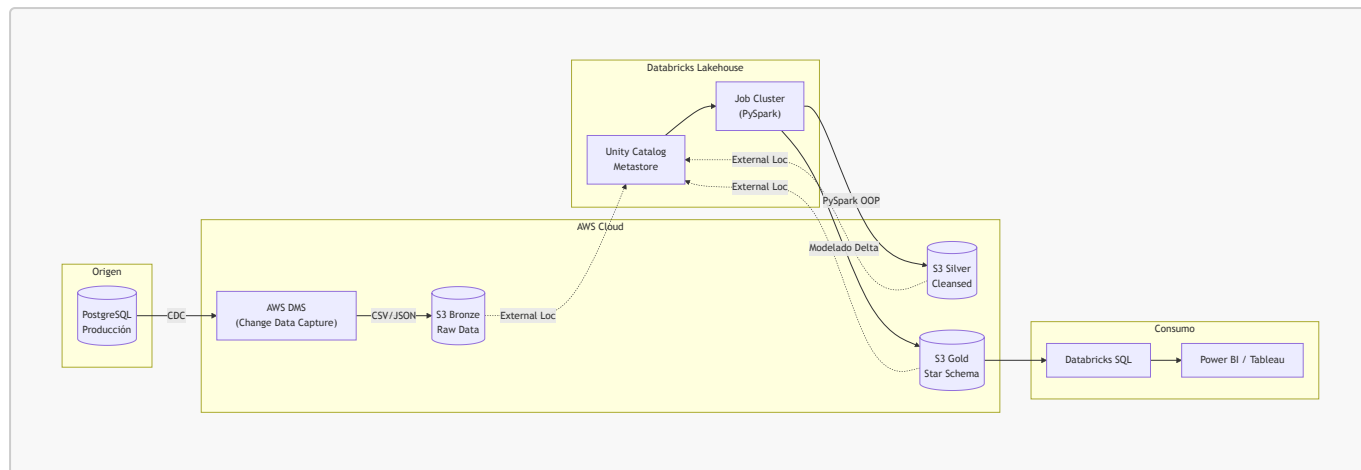
Script generador de datos transaccionales, creando perfiles de usuarios realistas para alimentar la capa Bronze y probar el rendimiento del pipeline.

2. Arquitectura de la Solución (End-to-End)

El flujo de datos implementado sigue el estándar de la industria conocido como **Arquitectura Medallion**, dividiendo los datos en tres capas de pureza:

- 🏅 **Capa Bronze (Datos Crudos):** Almacena los archivos exactos como llegaron de las fuentes transaccionales (`users`, `orders`, `order_items`). Es un respaldo inmutable histórico.
- 🥈 **Capa Silver (Datos Limpios):** Aquí la data cobra sentido comercial. Se tipifican las columnas, se eliminan registros corruptos y, **muy importante**, se encriptan (enmascaran) los datos personales de los clientes (PII) para cumplir con leyes de privacidad de datos.
- 🥇 **Capa Gold (Modelos de Negocio):** Los datos limpios se cruzan y consolidan en modelos para el negocio (*Star Schema*). Aquí nacen las tablas que usarán los analistas (ej. `dim_users` y `fct_sales`).

Diagrama General de Arquitectura Cloud



3. Gobernanza, Privacidad y Seguridad

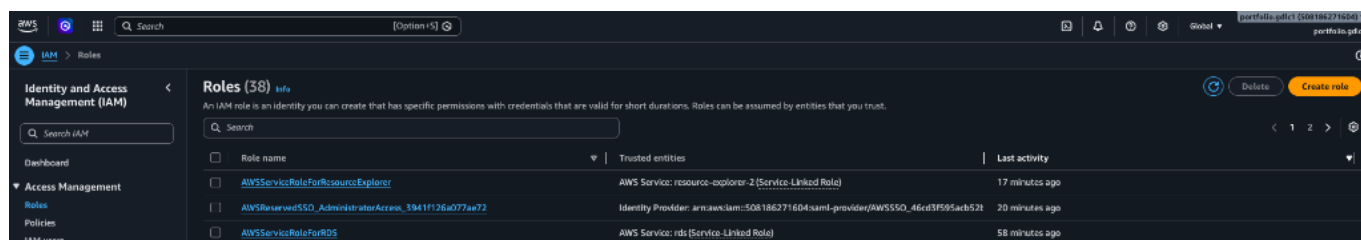
Este proyecto fue diseñado con la seguridad en primer plano, no como una idea de último momento:

1. **Protección de Datos Personales (PII):** Durante la transformación en la capa Silver (`st_users`), los correos electrónicos de los usuarios son encriptados mediante un algoritmo criptográfico de una sola vía (**SHA-256**). Esto permite a los analistas saber que es un usuario único, pero no pueden ver su correo real.
2. **Databricks Unity Catalog:** Actúa como el gran "gobernante" de los datos. Unity Catalog vincula las carpetas físicas de S3 con Databricks, permitiendo gestionar permisos a nivel de tablas, esquemas e incluso columnas.

🔑 Gestión de Accesos (AWS IAM Roles)

La arquitectura sigue el principio de **menor privilegio**, aislando las responsabilidades en diferentes Roles de IAM administrados por Terraform (todos prefijados con `spinbyoxxo-` para fácil identificación en tu consola de AWS):

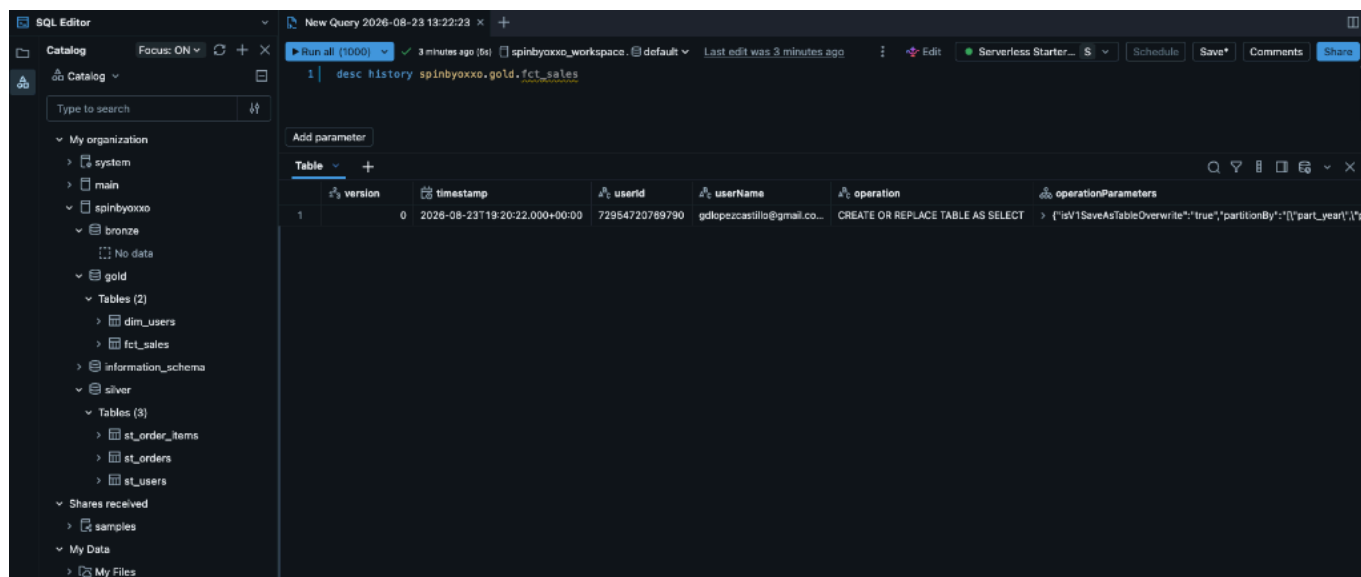
- **Cross-Account Role** (`spinbyoxxo-databricks-cross-account-role`): Permite al plano de control global de Databricks instanciar clústeres EC2 (las máquinas virtuales que ejecutan Spark) en tu VPC de AWS de manera segura usando `External ID`.
- **UC Metastore Access Role** (`spinbyoxxo-uc-access`): Rol exclusivo para que el Metastore de Unity Catalog pueda administrar sus metadatos internos en su bucket S3 dedicado (`spinbyoxxo-databricks-metastore-...`).
- **UC Storage Credentials Role** (`spinbyoxxo-uc-storage-role`): Rol crítico que cuenta con políticas `self-assuming` estrictas. Es utilizado por Databricks para leer y escribir los datos reales directamente en los Data Lakes (los buckets Bronze, Silver y Gold).



Políticas de IAM aprovisionadas desde código, garantizando la seguridad entre cuentas (Cross-Account).

Nota: En tu consola verás roles nativos de AWS (como `AWSServiceRoleForRDS`), pero los roles de esta arquitectura siempre inician con el prefijo del proyecto.

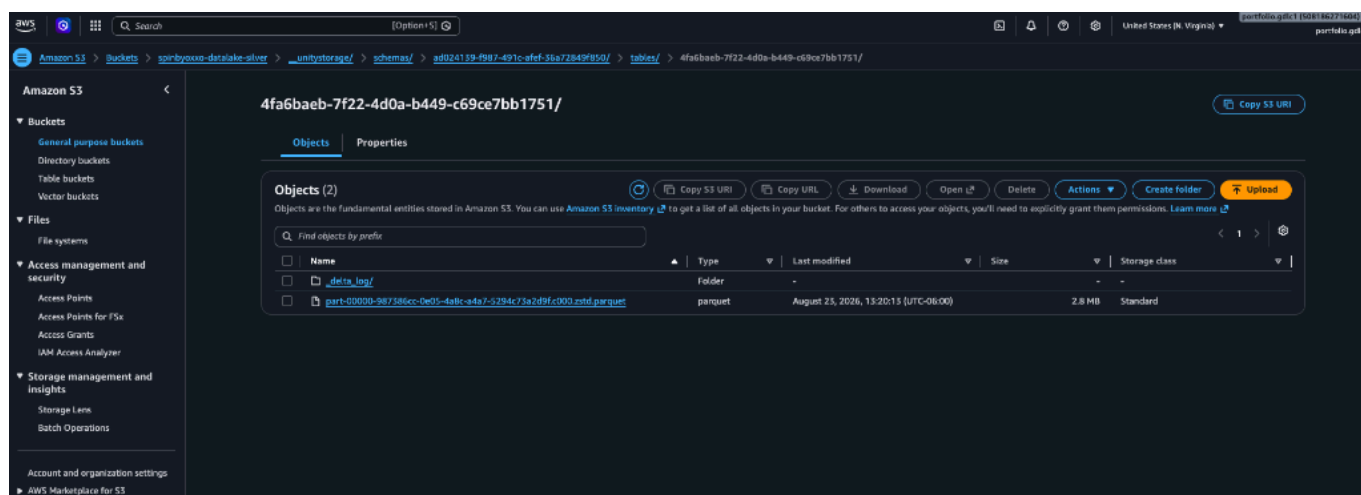
- Formato Delta Lake:** Todas las tablas de las capas Silver y Gold se guardan nativamente en formato **Delta**. Esto permite a la empresa hacer "viajes en el tiempo" (recuperar datos borrados) y asegura que, si un proceso de transformación falla a la mitad, los datos no se corrompan (transacciones ACID).



Demostración de las capacidades transaccionales de Delta Lake (`DESC HISTORY`), mostrando el registro de auditoría de quién y cómo se modificó la tabla.

Almacenamiento Físico en AWS S3

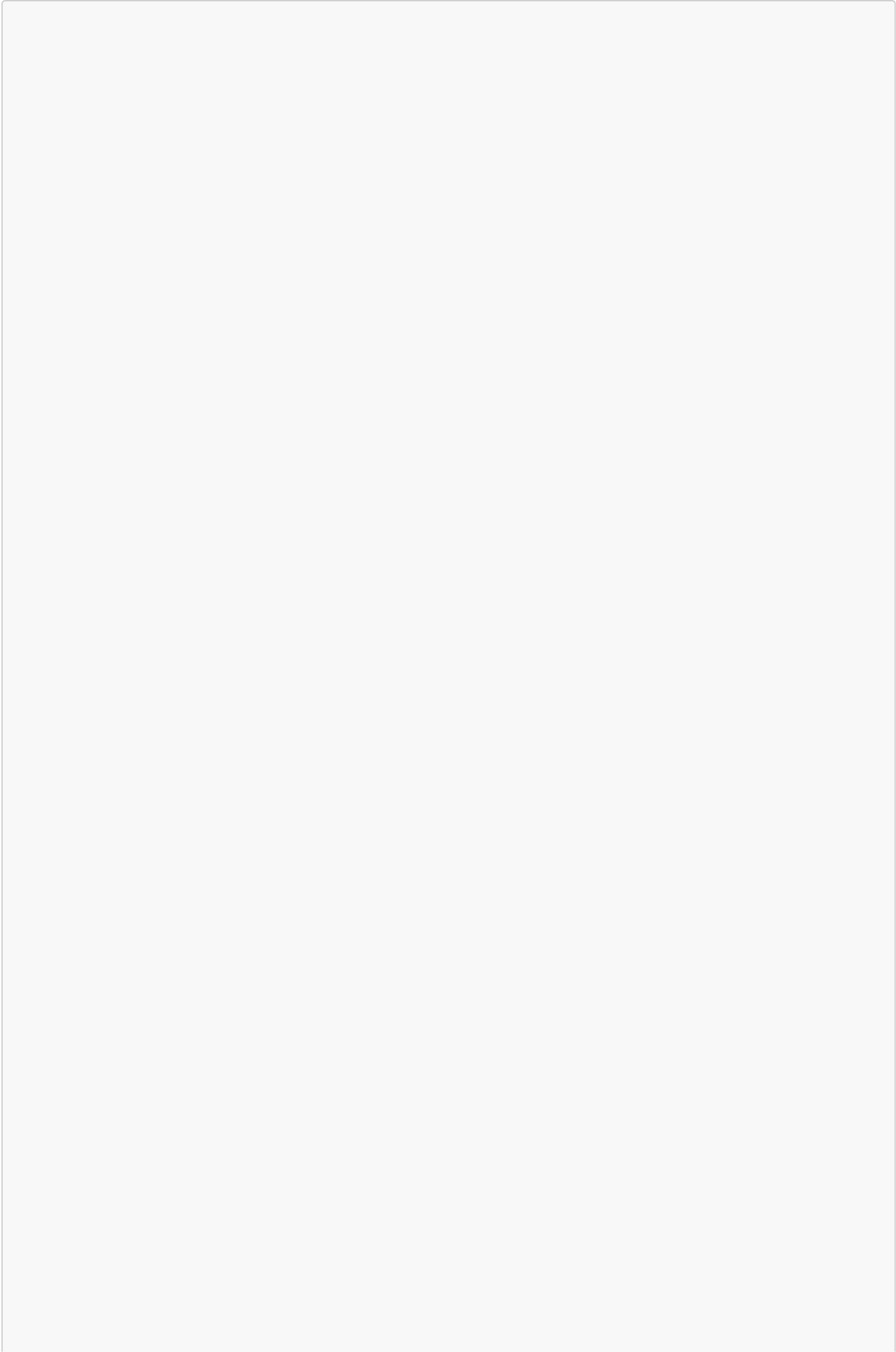
Aunque los analistas interactúan con tablas en Databricks, los datos reales residen de manera segura en tu propia cuenta de AWS. Unity Catalog abstrae la complejidad organizando los archivos transaccionales (`.parquet` y `_delta_log`) bajo identificadores únicos (UUIDs).



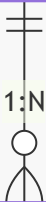
Almacenamiento nativo de las tablas Delta dentro del bucket S3 (Capa Silver), administrado y gobernado transparentemente por Unity Catalog.

Diseño de Datos (Capa Silver)

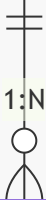
En la capa Silver los datos se limpian, enmascaran y tipifican, pero conservan la estructura transaccional original antes del modelado analítico:



st_users			
string	user_id	PK	
string	name		
string	email_hash		PII Enmascarada
string	country		
timestamp	created_at		
timestamp	updated_at		



st_orders		
string	order_id	PK
string	user_id	FK
decimal	total_amount	
string	status	
date	order_date	
timestamp	updated_at	

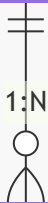


st_order_items		
string	item_id	PK
string	order_id	FK
string	product_id	
decimal	quantity	

	decimal	price	
--	---------	-------	--

Diseño del Modelo de Datos (Capa Gold)

dim_users		
string	user_id	PK
string	name	
string	email	
string	country	
timestamp	created_at	
timestamp	updated_at	
boolean	is_active	



fct_sales		
string	id	PK
string	order_id	
string	user_id	FK
string	product_id	
decimal	total_amount	
decimal	quantity	
decimal	price	
string	status	
date	order_date	
int	part_year	
int	part_month	

(La tabla `fct_sales` está particionada físicamente en el almacenamiento en la nube por `part_year` y `part_month` para acelerar drásticamente los reportes que buscan ventanas de tiempo específicas).

SCD Tipo 2 (Slowly Changing Dimensions): Como se aprecia en la tabla `dim_users`, se implementó el campo `is_active` (booleano). Esto prepara el modelo de datos para retener el historial completo de cambios de un usuario (por ejemplo, si cambia de país). Cuando un dato cambia, el registro antiguo se marca como inactivo (`is_active = False`) y se inserta el nuevo como activo, permitiendo que las ventas históricas de `fct_sales` apunten siempre a la fotografía correcta del usuario en el momento de la compra.

4. Ingeniería Híbrida: Código Limpio y Escalable

El código de transformación no es un script gigante difícil de leer. Fue diseñado combinando **Programación Orientada a Objetos (OOP)** y **Jupyter Notebooks**, logrando lo mejor de ambos mundos:

- **La Clase Maestra (`src/etl_master.py`):** Contiene la lógica dura y repetitiva. Se encarga de extraer, cargar en formato Delta y particionar, todo leyendo desde el `config.yaml`.
- **Los Notebooks Analíticos (`silver/*.ipynb`, `gold/*.ipynb`):** Cada tabla del negocio tiene su propio Notebook. Estos heredan de la clase maestra, por lo que su código es extremadamente limpio (solo declaran cómo se transforma el dato). Al ser Notebooks, los analistas de datos en la nube pueden abrirlos visualmente en Databricks y explorar las celdas sin tener que entender arquitecturas complejas de software.
- **Configuración Dinámica (`conf/config.yaml`):** Una sola "consola de mandos" que enciende, apaga o redirige tablas completas con cambiar una sola palabra, sin tocar el código fuente de Python.

```
! config.yaml X
spinbyoxxo-data-pipeline > conf > ! config.yaml
1  tables:
2    silver:
3      st_users:
4        active: 1
5        source_type: "csv"
6        source_path: "s3://spinbyoxxo-datalake-bronze/raw_postgres_users.csv"
7        catalog: "spinbyoxxo"
8        schema: "silver"
9        table_name: "st_users"
10       write_mode: "overwrite"
11       module_path: "silver.st_users.StUsers"
12       owner: "gerardodavidlopezcastillo"
13
14     st_orders:
15       active: 1
```

Un solo archivo YAML controla toda la orquestación: metadatos, esquemas (Silver/Gold), ubicaciones y modos de escritura.

5. Infraestructura como Código (Terraform)

Crear recursos de manera manual en Amazon Web Services (AWS) dando clics es una mala práctica propensa a errores humanos. Este proyecto cuenta con la carpeta `infrastructure/terraform/`, donde toda la infraestructura está automatizada en código. Con un solo comando (`terraform apply`), la nube crea de forma idéntica y segura:

- La red virtual (VPC, Subredes).
- Los contenedores S3 de almacenamiento.
- Los Permisos y Roles (IAM).
- El Workspace de Databricks y Unity Catalog.

```

infrastructure/terraform
├── .terraform
├── .terraform.lock.hcl
├── databricks_catalog.tf
├── databricks_iam.tf
├── databricks_network.tf
├── databricks_storage.tf
├── databricks_unity_catalog.tf
├── databricks_workspace.tf
├── network.tf
├── outputs.tf
├── provider.tf
├── s3.tf
├── terraform.tfstate
├── terraform.tfstate.backup
├── terraform.tfvars
└── variables.tf

Problems Output Debug Console Terminal Ports
(prjct_de_globalcase) + terraform git:(main) terraform apply
Apply complete! Resources: 2 added, 4 changed, 2 destroyed.

Outputs:
databricks_workspace_url = "https://dbc-8d53a6ea-4124.cloud.databricks.com"
processed_bucket_name = "spinbyoxxo-datalake-silver"
raw_bucket_name = "spinbyoxxo-datalake-bronze"
(prjct_de_globalcase) + terraform git:(main) terraform apply
data.databricks_aws_assume_role_policy.this: Reading...
data.databricks_aws_crossaccount_policy.this: Reading...
data.databricks_aws_assume_role_policy.this: Read complete after 0s [id=815efla5-3a14-4b5e-8e5a-0691d526e07a]
data.databricks_aws_crossaccount_policy.this: Read complete after 0s [id=]
data.databricks_user.me: Reading...
data.databricks_user.me: Read complete after 0s [id=72954720769790]
data.aws_iam_policy_document.passrole_for_uc: Reading...
aws_sip_nat_eip: Refreshing state... [id=eipalloc-0318cc408525db099]
aws_vpc.main: Refreshing state... [id=vpc-0b6a7d6d2305e0a07]
aws_iam_role.uc_storage_role: Refreshing state... [id=spinbyoxxo-uc-storage-role]
aws_iam_role.cross_account_role: Refreshing state... [id=spinbyoxxo-databricks-cross-account-role]
aws_s3_bucket.metastore: Refreshing state... [id=spinbyoxxo-databricks-metastore-508186271604]
aws_s3_bucket.raw_data: Refreshing state... [id=spinbyoxxo-datalake-bronze]
aws_s3_bucket.processed_data: Refreshing state... [id=spinbyoxxo-datalake-silver]
aws_s3_bucket.root_storage_bucket: Refreshing state... [id=spinbyoxxo-databricks-root-bucket-508186271604]
aws_s3_bucket.athena_results: Refreshing state... [id=spinbyoxxo-athena-results]

```

Despliegue de los recursos ejecutando `terraform apply` desde la terminal integrada, mostrando la organización modular de los archivos `.tf`.

Nota Técnica de Optimización: En iteraciones previas se probaron tecnologías antiguas (AWS Redshift, AWS ECS Fargate, ECR Docker). Al migrar a Databricks, se depuraron y destruyeron todos esos recursos obsoletos, demostrando un enfoque **FinOps** (optimización de costos) para no dejar infraestructura fantasma consumiendo presupuesto.

6. Guía de Ejecución Rápida

La mayor ventaja de esta arquitectura es su **Modelo Dual**. Puedes probar tu código localmente sin gastar dinero en la nube, y luego subir el mismo código a producción sin modificar una sola línea.

A) Ejecución Local (Para Desarrolladores / Pruebas)

1. Instala el entorno virtual con las dependencias:

```
pip install -r requirements.txt
```

2. Ejecuta el orquestador principal con la bandera de pruebas:

```
python main.py --onpremise
```

¿Qué hace la bandera `--onpremise`? Intercepta la configuración, simula la existencia de Unity Catalog dentro de tu computadora creando la base de datos `spark-warehouse`, y lee los CSV locales en la carpeta `bronze/` en lugar de ir a buscarlos al S3 de AWS.

```

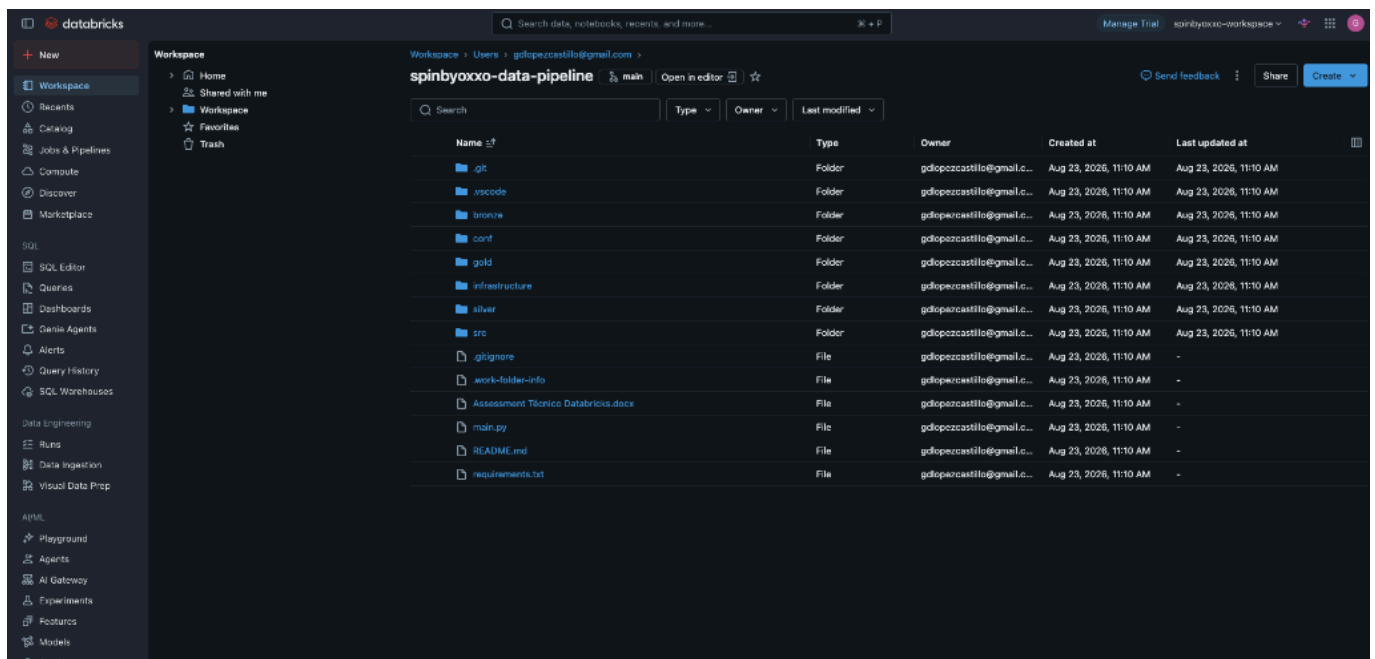
$ terraform init
$ terraform apply
$ python main.py --onpremise
26/08/23 13:11:14 WARN Utils: Your hostname, MacBook-Pro-de-user.local resolves to a loopback address: 127.0.0.1; using 192.168.1.213 instead (on interface en0)
26/08/23 13:11:14 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
:: Loading settings :: url = jar:file:/opt/homebrew/cask/ruby/bin/caskenv/prjct_de_globalcase/Lib/python3.10/site-packages/pyspark/jars/ivy-2.5.1.jar/org/apache/ivy/core/settings/ivysettings.xml
ivy Default Cache set to: /Users/user/.ivy2/cache
The jars for the packages stored in: /Users/user/.ivy2/jars
io.deltalake:spark-2.12 added as a dependency
:: resolving dependencies :: org.apache.spark:spark-submit-parent-95d47d81-8b78-4710-aac8-b57e9e2a3aa:1.0
:::
::: found io.deltalake:spark-2.12:3.1.0 in central
::: found io.deltalake:storage-3.1.0 in central
::: found org.apache.spark:spark-submit-parent-95d47d81-8b78-4710-aac8-b57e9e2a3aa:1.0 in central
::: resolution report :: resolve 87ms :: artifacts dl 5ms
:::
::: modules in scope
:::
conf | number | modules | artifacts
--- | --- | --- | ---
default | 3 | 0 | 0 | 0 | 3 | 0
::: retrieving :: org.apache.spark:spark-submit-parent-95d47d81-8b78-4710-aac8-b57e9e2a3aa
:::
::: 0 artifacts copied, 3 already retrieved (804/3ms)
26/08/23 13:11:14 WARN NativeCodeLoader: Unable to load native-udao library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
INFO: main --- Cargando config.yaml...
INFO: main --- Inicializando tabla: st_users ---
INFO:StUsers:Extrayendo datos (Tipo: csv)
INFO:src.analytics.util:Se aplicó a 6 columnas y 10000 filas
INFO:StUsers:Guardando en Delta Table: spark_catalog.silver.st_users | Data Owner: gerardodavidlopezcastillo
26/08/23 13:11:20 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
INFO: main --- Inicializando tabla: st_order_items ---
INFO:StOrderItems:Extrayendo datos (Tipo: csv)
INFO:src.analytics.util:Se aplicó a 6 columnas y 30000 filas
INFO:StOrderItems:Guardando en Delta Table: spark_catalog.silver.st_order_items | Data Owner: gerardodavidlopezcastillo
26/08/23 13:11:27 WARN MemoryManager: Total allocation exceeds 50.00% (1,023,834,720 bytes) of heap memory
Scaling row group sizes to 95.00% for 8 writers
INFO: main --- Inicializando tabla: gld_sales ---
INFO:GldSales:Extrayendo datos (Tipo: table)
INFO:GldSales:Guardando en Delta Table: spark_catalog.gold.gld_sales | Data Owner: gerardodavidlopezcastillo
INFO: main --- Inicializando tabla: fct_sales ---
INFO:FctSales:Calculando métricas de ventas (Fact Table)
INFO:FctSales:Guardando en Delta Table: spark_catalog.pold.fct_sales | Data Owner: gerardodavidlopezcastillo
INFO:FctSales:Aplicando particiones por: ["part_year", "part_month"]

```

Ejecución exitosa del pipeline de manera local mediante el flag `--onpremise`, simulando las escrituras en una base de datos local embebida.

B) Ejecución en Databricks (Cloud Producción)

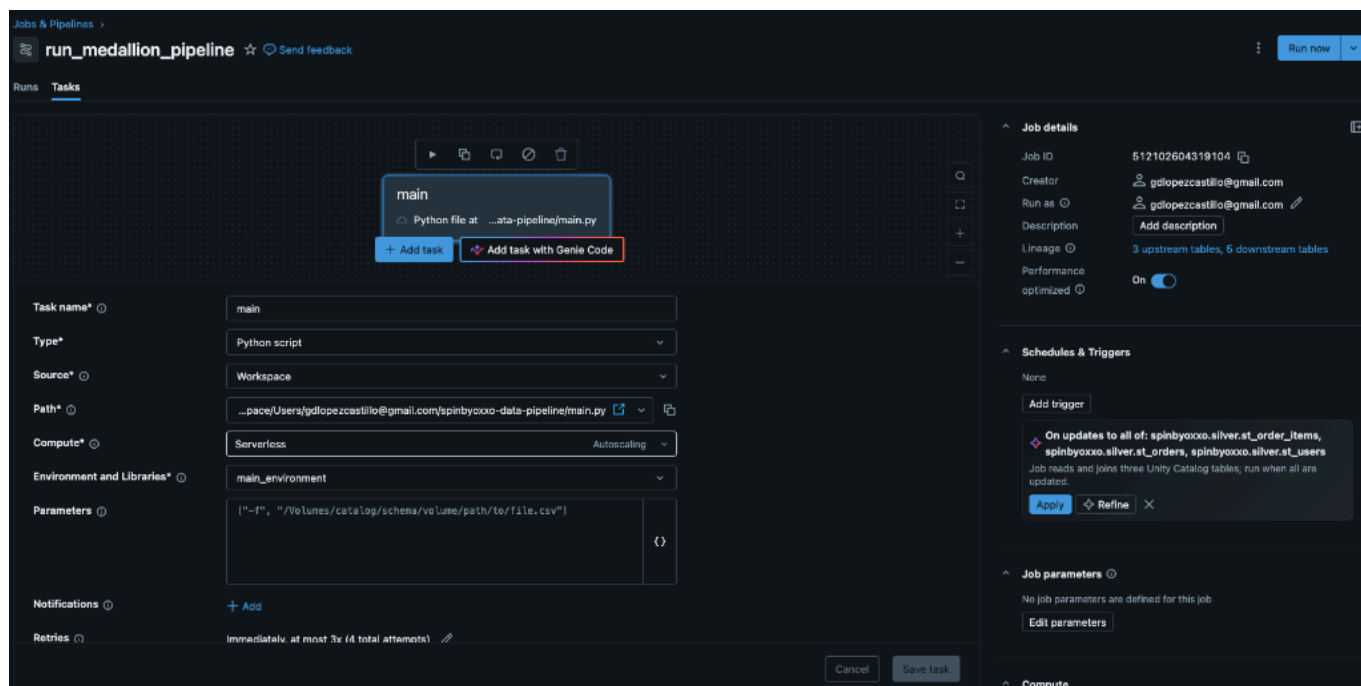
1. En tu espacio de trabajo (Workspace) de Databricks, usa la función **Databricks Repos (Git Folders)** para conectar este repositorio de GitHub.
2. Databricks clonará el proyecto idéntico a la nube.



Workspace de Databricks sincronizado con el repositorio de GitHub de manera nativa para CI/CD continuo.

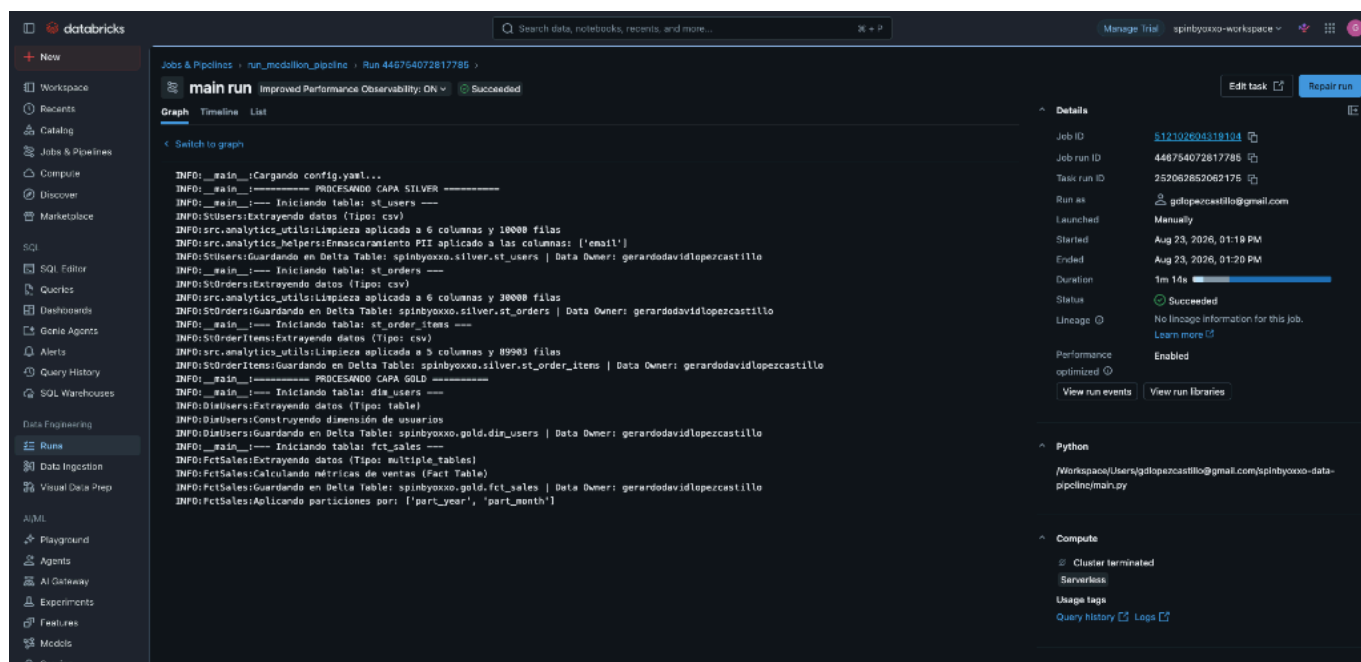
3. Dirígete a la pestaña **Workflows -> Create Job**.
4. Crea una tarea de tipo **Python Script** apuntando al archivo `main.py` de tu repositorio recién sincronizado.

5. En la configuración del Clúster del Job, asegúrate de instalar las librerías **omegaconf** y **ipynb**.



Configuración de la tarea en Databricks Workflows apuntando al script principal y definiendo los parámetros de cómputo serverless.

6. Presiona **Run Now**. Al no enviar la bandera **--onpremise**, el script utilizará los recursos empresariales: leerá los terabytes de datos en AWS S3 y guardará las tablas en Unity Catalog.



Ejecución 100% exitosa del pipeline completo orquestado mediante un Job Cluster automatizado.

Consumo de Datos (Databricks SQL)

Una vez completado el pipeline, la arquitectura permite de manera instantánea realizar analítica de datos utilizando Databricks SQL Editor o conectándolo con herramientas de BI (PowerBI, Tableau, etc.).

The screenshot shows the Databricks SQL Editor with a query executed: `select * from spinbyxoxo.gold.fct_sales limit 100`. The results table displays columns: order_id, user_id, product_id, date_sk, status, quantity, price, and is. The data includes various order statuses like 'Cancelled', 'Completed', 'Shipped', and 'Pending'.

order_id	user_id	product_id	date_sk	status	quantity	price	is
6a6a3f3f-83a1-d332-b6b0-7784ba3cafa	184ba6b0-ac39-4cfa-926f-2058a641a9d	PROD-135	2026-05-15	Cancelled	4	34.13	
4506b248-efbd-4211-8ef9-bf85356603bb	76d9b2f2-10c6-48db-8bd1-d9d0e2133336	PROD-312	2026-06-12	Completed	1	82.59	
c2323f3c-d8c4-4e8b-87b0-64da8b908f2	51c0b3f1-1264-4125-979f-4c366cc48af4	PROD-673	2026-06-29	Cancelled	4	18.68	
38b5297f-2d41-47e7-b122-a514ade24c38	fecb228f-2770-4e18-8e01-bed22847d086	PROD-309	2026-05-07	Completed	2	139.79	
cf302830-bf81-4d35-b240-6a54c96047d	f9778beb-57cb-4b85-8ebc-c5455192fb13	PROD-721	2026-05-03	Shipped	1	128.11	
85cf80a7-68fa-4d5d-bb04-7f8f31fa7dd8	915e719e-3d76-4ecf-a01b-b663a6a8da2	PROD-863	2026-05-15	Completed	2	12.40	
a8d0b4c6-7b32-475d-ac40-5038d389d7d7	a8f934bc-a1cc-4d42-a2df-c7910e7cc07c	PROD-733	2026-06-06	Shipped	2	141.88	
71ad86b6-f92b-4aba-8a53-cf04f88877d	0aa40218-726f-44ca-ba29-3d11d8c07e57	PROD-829	2026-05-15	Completed	4	111.59	
aa8e1129-9cc5-478c-af16-1d31bb21389d	71b994c1-4af2-43a0-baf6-1b74f4cb9883	PROD-249	2026-05-11	Pending	3	72.23	
8802055b-926f-4d09-ba75-f306c653abfd	25a21ba9-f15a-45b6-b6d0-61d202a3c4a	PROD-781	2026-05-22	Completed	1	63.21	
38a8f8b8-1151-4150-9e45-727d00a0ee0	c81967cd-ecfa-4da8-a726-940a39049e2	PROD-642	2026-05-11	Completed	3	126.32	
75cd6586-c69b-4766-8cd3-395d79d7f6c2	94bacc28-bd16-4a37-9d25-cf3c3c6dbdb	PROD-416	2026-05-26	Completed	3	83.36	
75cd6586-c69b-4766-8cd3-395d79d7f6c2	94bacc28-bd16-4a37-9d25-cf3c3c6dbdb	PROD-806	2026-05-26	Completed	1	52.53	
221b98b0-802a-449b-b66f-73372aa27772	21c0e873-771b-4944-b765-d7c03c6d89c	PROD-983	2026-05-22	Completed	4	138.09	
433af3ca-faa4-4268-8346-ae8acc633fcd	3cfc4fcd-74aa-47a5-8a52-8cda3e14bda	PROD-750	2026-05-25	Completed	4	118.93	
e0a9c9a6-467e-4b3e-96a2-cbc39d32817d	87ab922c-07eb-4302-b89e-b999f1998851	PROD-949	2026-05-10	Completed	4	58.69	
2c8aebdc-bc44-4e02-80f7-6704dcb3e994	a8c99326-5d54-4e4d-b895-2dc9777b219a	PROD-622	2026-05-07	Pending	3	16.44	
53c8c339-bd85-4d2a-88a1-487eb2ae5d2d	5193e342-98f6-4e8a-9e0a-b8eac9345f8	PROD-249	2026-05-07	Completed	1	14.38	
83a74ccf-fccb-465f-950c-b5da7d95ebcc	92b29047-073a-4c20-bc4c-eed338f6d5c6	PROD-150	2026-05-08	Completed	3	102.85	
38d8f0a4-7014-4d39-8bc3-e04a3b55a11	9157d0a4-711b-4168-ac7e-bbc396f3dbaf	PROD-701	2026-05-17	Cancelled	1	120.54	
5209d9d1-c409-4866-bc52-788d3724e683	9cd99d73-000c-40bc-a2bc-3dca0f4b5780	PROD-467	2026-05-04	Pending	1	79.43	
cab70b86-771b-45fb-a52e-5ba7c0f0e14	8b5b0a65-9f3b-4d48-833b-a67c7f85aeb	PROD-748	2026-05-02	Pending	4	7.77	

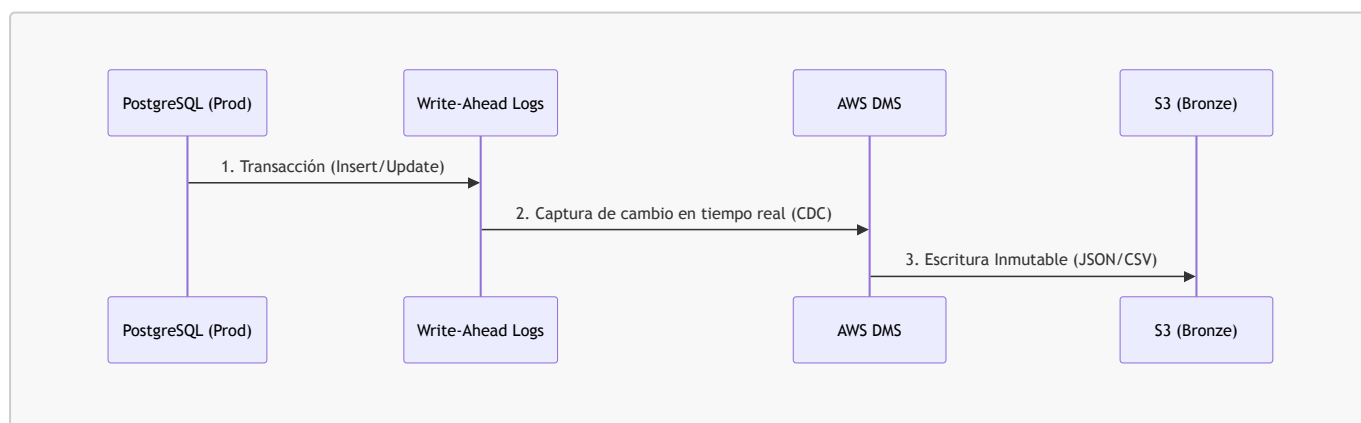
Consulta de las tablas de negocio finales de la capa Gold utilizando Databricks SQL Engine.

7. Defensa de Decisiones Técnicas (Respuestas al Caso de Negocio)

Como parte de la sustentación técnica de esta arquitectura, a continuación se detallan las respuestas a los requerimientos de negocio planteados:

1. Ingesta: ¿Batch o Streaming (CDC)? ¿Cómo mitigar el impacto en Producción?

Decisión: Se implementa un enfoque de **Change Data Capture (CDC)** mediante **AWS DMS (Database Migration Service)**. **Justificación:** Al ser una base de datos OLTP (PostgreSQL) crítica para la app móvil, realizar consultas masivas tipo **SELECT** * para extracciones Batch en horas pico degradaría el servicio. AWS DMS lee directamente de los **Write-Ahead Logs (WAL)** de PostgreSQL, extrayendo los cambios (Inserts/Updates/Deletes) en tiempo real con un impacto casi nulo en el rendimiento de la base de datos origen. Estos eventos aterrizan como archivos crudos inmutables en la capa **Bronze (S3)**.



2. ¿Cómo asegurar que los datos sensibles (PII) estén protegidos?

Decisión: Enmascaramiento criptográfico y aislamiento de permisos. **Justificación:** Durante el salto de la capa Bronze a la capa Silver (ver `src/analytics_helpers.py`), los campos como el **email** de los usuarios pasan por una función Hash unidireccional (**SHA-256**). De esta forma, el dato original se destruye para los analistas, pero permite seguir trazando la identidad del usuario (ej. para saber cuántas

compras hizo). Adicionalmente, el acceso a la capa Bronze queda estrictamente prohibido para analistas a través de **Unity Catalog**, permitiendo solo a roles de Ingeniería ver el dato crudo.

```
class AnalyticsGobernanzaHelpers:

    @staticmethod
    def gobernanza_validar_nit_boolean(df: DataFrame, *cols) -> DataFrame:
        return df

    @staticmethod
    def enmascarar_pii_sha256(df: DataFrame, columnas_pii: list) -> DataFrame:
        """
        Aplica un hash SHA-256 a columnas sensibles (ej. emails, teléfonos)
        para cumplir con normativas de seguridad (GDPR/Data Privacy).
        """
        df_masked = df
        for col_name in columnas_pii:
            if col_name in df_masked.columns:
                df_masked = df_masked.withColumn(
                    f"{col_name}_hash",
                    F.sha2(F.col(col_name).cast("string"), 256)
                ).drop(col_name) # Eliminamos la columna original cruda

        logger.info(f"Enmascaramiento PII aplicado a las columnas: {columnas_pii}")
        return df_masked
```

Fragmento de código en PySpark mostrando la función SHA-256 aplicada a columnas dinámicas configuradas.

3. Estrategia de particionamiento para tablas de hechos (Fact Tables)

Decisión: Particionamiento físico por `part_year` y `part_month`. **Justificación:** El comercio electrónico genera un volumen masivo de transacciones. En la capa Gold, la tabla `fct_sales` está particionada por año y mes. Cuando un analista financiero ejecute reportes como "Ventas totales en el último trimestre", el motor de Databricks SQL ignorará el 90% de los datos históricos irrelevantes (Partition Pruning), leyendo solo los meses necesarios. Esto ahorra tiempos de cómputo y costos drásticamente.

```
1 | desc table spinbyoxxo.gold.fct_sales
```

Add parameter

Table +

	^A _C col_name	^A _C data_type	^A _C comment
1	order_id	string	null
2	user_id	string	null
3	product_id	string	null
4	date_sk	date	null
5	status	string	null
6	quantity	int	null
7	price	decimal(10,2)	null
8	line_total_amount	decimal(22,2)	null
9	part_year	int	null
10	part_month	int	null
11	# Partition Informati...		
12	# col_name	data_type	comment
13	part_year	int	null
14	part_month	int	null

Descripción de la tabla

fct_sales en Unity Catalog confirmando que los datos están particionados por año y mes.

4. Cambios de estado (ej. Pending a Completed): ¿Cómo se refleja en la arquitectura?

Decisión: Operaciones **UPSERT** gracias a las propiedades ACID de Delta Lake. **Justificación:** En un Data Lake tradicional (solo Parquet), actualizar un registro es extremadamente ineficiente (requiere reescribir todo el archivo). Como usamos **Delta Lake** en las capas Silver y Gold, podemos utilizar la operación **MERGE INTO**. Cuando el CDC captura que una orden cambió a **Completed**, Spark ejecuta un *Upsert* sobre la tabla Delta: si el ID de la orden existe, actualiza el estado; si no existe, la inserta. Así, los dashboards siempre reflejan el estado real sin duplicar registros.

5. ¿Qué framework de CI/CD implementarías y cómo manejarías laC?

Decisión (Diseño Teórico): **GitHub Actions** orquestando **Terraform** y Databricks CLI. **Justificación:** Dado que este proyecto ya cuenta con el código de laC y Pipelines, la evolución natural empresarial (no implementada en el código actual, pero diseñada) se divide en dos fases:

Integración Continua (CI):

- Al hacer un Pull Request hacia **main**, un Workflow de GitHub Actions ejecutará un linter (ej. **flake8**) y tests unitarios locales (ej. usando **pytest** llamando a **main.py --onpremise**).
- Se ejecutará **terraform plan** para evaluar qué infraestructura va a cambiar en AWS/Databricks, entregando el plan como un comentario en el PR para revisión manual.

Despliegue Continuo (CD):

- Al hacer merge a la rama **main**, un job ejecuta **terraform apply -auto-approve** creando o modificando recursos (S3, IAM, Unity Catalog).
 - Posteriormente, a través de la API de Databricks, se dispara una actualización al *Git Repo* del Workspace para sincronizar el código PySpark.
 - Finalmente, se despliegan/actualizan los *Databricks Workflows (Jobs)* usando Databricks Asset Bundles (DABs) o Terraform, asegurando que la infraestructura y el código estén siempre alineados.
-

Gerardo López

Senior Data Engineer | Data Architect | Cloud Engineer

